# 1 MOC configuration settings

## Overview

A large number of configuration settings specify the layout and functions of the MES Operation Center (MOC): this includes, for example, the current window size, the language used or the number and order of columns in a table of a specific application. This section provides background information on the management of the MOC configuration settings and describes how custom configuration settings can be shared in the entire company.

## 1.1 Configuration settings and configuration levels

Every configuration setting may have up to four different values subject to the currently valid configuration level (scope). MOC has the following configuration levels (scopes):

- The “standard” configuration level (scope) includes values that MPDV provides for all users.
- The “custom” configuration level (scope) includes customized values that MPDV provides for all users.
- The “local” configuration level (scope) includes customized values that are provided locally for all users (e.g. by an administrator or key user).
- The “user” configuration level (scope) includes the values that a user has created individually.

At runtime, the MOC imports all values and selects the most specific value. If the “user” configuration level (scope) includes a value, this one will be used instead of the values from the levels “local”, “custom” or “standard”. This rule always applies up to the currently active configuration level, i.e. if the "Local" level is active, only values from the "Local", "Custom" or "Standard" levels are used, but not from the "User" level.

Use the “local” configuration level (scope) to make global configurations intended for the use throughout the entire system.

If you want to make changes in the “local” configuration level (scope), activate the “local scope” at first. Then all changes, for example, to applications are written in the scope folder after saving, i.e. in the subfolder custom\conf of the MOC program directory.

## 1.2 Activation of a configuration scope

In general, the MOC is operated with the configuration scope “user”.

Use the system option “DefaultSettingScope” or the function “MES Development Suite", "System Information Center” to set the configuration scope. (Note: the latter function is only available if corresponding licenses have been purchased.)

Set the system option “DefaultSettingScope“ in the file MOC.ApplicationSettings.config or via a command line parameter.

The following row in the file MOC.ApplicationSettings.config specifies the option:

```
<add key="DefaultSettingScope" value="User" />.
```

Allowed values are “standard“, “local“, “custom" and “user". Restart the MOC, once you have made any changes.

As an alternative, set the configuration scope via the command line parameter DefaultSettingScope=<scope> (e.g. in a link to moc.exe). Example

```
C:\Programme\MOC.exe DefaultSettingScope=Local
```

Only MPDV staff are permitted to use the configuration scopes “standard” and “custom”. Normally, users do not need to make changes in these configuration scopes, as this would endanger system stability and, in particular, the system’s ability to be upgraded.

## 1.3 Storage locations for configuration settings

All configuration values are stored in files, whereby a file usually combines a whole series of configuration values. The files of a configuration level are always compiled in a folder structure. By default, the following paths are used:

- The configuration values of the “user” configuration level (scope) are filed in the subfolder user\conf of the Windows user folder of the MOC application. You can find the folders here:
  Windows 7: [LocalApplicationData]\MPDV\MOC\user\conf
- The configuration values of the “local” configuration scope are stored in the subfolder *local\conf* of the MOC program directory.
- The configuration values of the “custom” configuration scope are stored in the subfolder *custom\conf* of the MOC program directory.
- The configuration values of the “standard” configuration scope are filed in the subfolder *conf* of the MOC program directory.

You can find the currently applicable storage locations via the MOC function “Help" => "System information” => “System” tab => “Configuration scopes”

You can change the storage locations for configuration scopes by configuring the system options in the file MOC.ApplicationSettings.config. The sections that follow describe these options.

Note that the MOC update process only uses the standard values for storage locations. If you change the storage locations, you are responsible for making sure that software updates are also installed in the new folders.

If you change the storage location, please note that the application start might be slowed down when files are loaded via network.

### 1.3.1 User data

The system option “UserDataDirectory” specifies where user data, i.e. the configuration values changed by the user are stored. You can use placeholders when you define paths.

Default value:

```
<add key="UserDataDirectory" value="$ApplicationData\user\" />
```

Note: $ApplicationData refers to the data directory of the MOC application. In Windows 7 this is the folder C:\Users\<user>\AppData\Roaming\MPDV\MOC\.

Allowed placeholders are:

- %HYDRAUSER%: name of the logged user
- %WINDOWSUSER%: the name of the registered Windows user
- %HYDRASYSTEM%: the name of the system the user is logged on to.

Example:

```
<add key="UserDataDirectory" value="\\dataServer\moc\users\%hydrauser%\" />
```

### 1.3.2 System-wide (local) changes

The system option “LocalConfigurationDirectory” determines where configuration data of the “local” configuration scope is stored. This configuration scope includes individual or local changes applicable to the entire system.

Example:

```
<add key=" LocalConfigurationDirectory" value="\\dataServer\moc\local\" />
```

Note: In this case, you cannot use the placeholders described in section 1.3.1!

### 1.3.3 Customizations by MPDV

The "CustomConfigurationDirectory" system option controls where the configuration data of the "Custom" configuration level is stored that contain the customizations provided by the MPDV.

Example:

```
<add key=" CustomConfigurationDirectory" value="\\dataServer\moc\custom\" />
```

Note: In this case, you cannot use the placeholders described in section 1.3.1!

### 1.3.4 Notes on application configurations

Each application has a separate subfolder for configuration files in the subfolder conf\MOC\Apps of the corresponding “scope folder”. For example, the settings of the application " Workplaces / Resources" with the application ID "workplaceoverview" are located in the following folders:

| Configuration scope | Folder |
|---|---|
| User | C:\Users\<cbu>\AppData\Roaming\MPDV\MOC\user\conf\Moc\Apps\WorkplaceOverview |
| Local | C:\Programme\MPDV\MOC\local\conf\MOC\Apps\WorkplaceOverview |
| Custom | C:\Programme\MPDV\MOC\custom\conf\MOC\Apps\WorkplaceOverview |
| Standard | C:\Programme\MPDV\MOC \conf\MOC\Apps\WorkplaceOverview |

The scope of a configuration value depends on the folder. Consequently, you can also transfer changed values to the “local scope” by copying files from the "user" directory to the "local" directory.

If you click on the save function in an application, only the changes made are saved. This means that the folder of the “local” configuration scope does not include the entire application, but only the contents deviating from the “standard” configuration scope.

If you load configurations, the folder that includes the settings file determines the configuration scope of a configuration value. If you copy files from a "user" directory to the "local" directory, you can transfer changed values to the "local" configuration scope.

## 1.4 Distribution of configuration settings

To deploy an application configuration changed in the “local scope” to other MOC installations, copy this configuration to the folder of the “local” configuration scope of the required installations.

1. Save the changes in your MOC installation.
2. Create update package:
   Use the MOC Update Package Creator to create an update package and to deploy the new configuration. Start the MOC Update Package Creator via the MOC function *Extras* → *Generate update package.*
3. Install update package on the server:
   Use the Maintenance Manager to install the created update package.
4. Update the other MOC installations:
   Use the MOC updater to update the MOC clients.

You can find further information in the sections dealing with MOC Update Package Creator and Maintenance Manager.

## 1.5 Configure syntactic types

### Overview

| | |
|---|---|
| Menu | - |
| Transaction code | syty |
| Function authorization | syty |

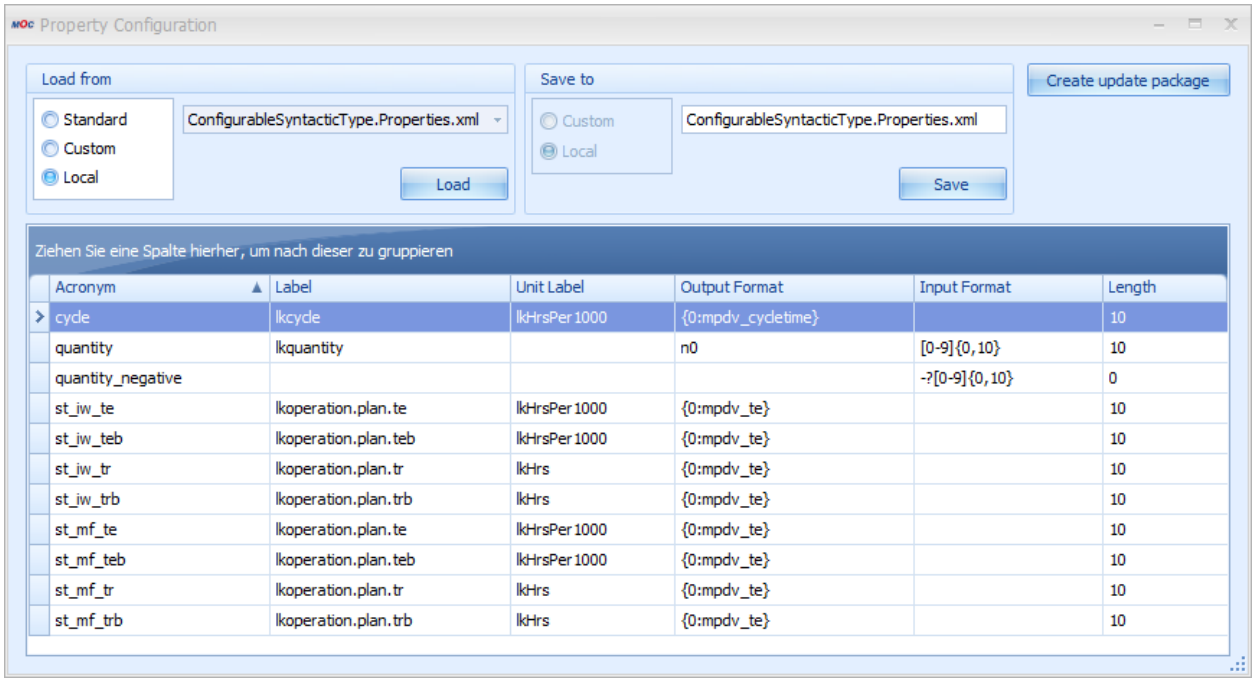
Syntactic types control at the MOC the standardized presentation of data at all locations via a central definition. Consequently, you can use the syntactic type for quantities to specify the number of decimal places. This setting affects all quantities displayed in the MOC.

Most syntactic types can only be changed by MPDV or customers/partners if they use the MES Development Suite.

However, the application "Configure syntactic types" additionally offers the option to configure selected important syntactic types directly on the customer system without having to order customizations or use the MES Development Suite.

The application "Configure syntactic types" is an expert application and only available in English. Usually, MPDV consultants use this application to change specific syntactic types of MOC data according to the customer's requirements.

The application "Configure syntactic types" uses the methods of the MES Development Suite. The document "MES Development Business Applications & Services“ (MDS-BAS)" provides further basic information on the MES Development Suite. You require this knowledge when working with the application "configure syntactic types".




This document illustrates the procedure step by step. Consequently, even unexperienced users should be in the position to make the most important changes independently.

You can configure the syntactic types included in the client file *ConfigurableSyntacticType.Properties.xml.*

## Definition of terms

Technical terms from the MES Development Suite and system administration are used here in connection with this application.

**Scope**

A scope describes a level at which configuration and programming can be performed in the system:

**Standard**

MPDV uses the *standard* scope to deliver standard products.

**Custom**

MPDV uses the *custom* scope to deliver customizations that complement or overwrite the standard.

**Local**

Customers or partners can use the *local* scope to make changes that complement or overwrite the *custom* and the *standard* scope.

**Update package**

An update package includes programs and configurations the Maintenance Manager first installs on the server. Then the MOC updater distributes the MOC data from the server to the other MOC clients. Usually, this is an automatic process.

**Update Package Creator**

The Update Package Creator is a tool used with the MOC to pack locally created customizations in an update package. The application "configure syntactic types" uses a simplified and restricted version of the Update Package Creator.

**Maintenance Manager**

Web application used to install update packages on the server. Usually, your IT department is familiar with the procedure as MPDV regularly sends updates that are installed via the Maintenance Manager.

## Overview of the procedure

This section provides a brief overview of the steps you have to carry out to change syntactic types. The sections that follow provide further details.

1. Load configuration: load the existing configuration of syntactic types from the system.
2. Change table data: change the configuration.
3. Save the changes.
4. Create update package: create an update package to distribute the new configuration.
5. Install update package: use the Maintenance Manager to install the update package.
6. Update MOC clients: use the MOC updater to update the MOC clients.

## 1) Load configuration

Load the syntactic types from the system before you can change them. You can select the scope:

**Local**

Loads the syntactic types you have already changed. In case you have not changed the syntactic types, the system loads the syntactic types provided by MPDV from the *custom* scope or the *standard* scope.

**Custom**

Loads the changed syntactic types provided by MPDV. In case MPDV has not changed the syntactic types, the system loads the standard configurations. This process does not include the syntactic types you have already changed.

**Standard**

Loads the syntactic types from the standard scope. This process does neither include the customizations provided by MPDV nor the changes you made.

As a general rule, you should choose the following methods for loading syntactic types:

- Create or change the customizations you made: load configurations from the *local* scope.
- Discard the changes you made and reset configurations to the version delivered by MPDV: load configurations from the *custom* scope.

## 2) Change table data

You can make changes to the table. In most cases only changes in the columns *OutputFormat, InputFormat* and *Length* are required.

## Table columns

**Acronym**

Name of the syntactic type. You should not change this value.

**Label**

Labeling of input fields displayed in front of the input fields. By default, "language keys" are used for the labels. These keys are translated depending on the language set in the MOC. If required, you can also enter customer-specific texts instead of "language keys". But these texts will not be translated. Customers using the product MES Development Suite (MDS-BAS) can define their own language keys. There are some rare places in the MOC that are not affected by changed labels. But the entire system will be affected if you use the MES Development Suite (MDS-BAS) to customize the *language key* of the label.

**UnitLabel**

The UnitLabel is the labeling displayed behind the input field. The same rules apply as for the label.

**OutputFormat**

Output format for data. A separate section describes expedient output formats.

Times and durations are internally stored in the system as integer seconds. If you convert times or durations during input or output formatting to formats other than hours, minutes and seconds (HH:MM:SS), the conversion may not be possible without losses. For example, this applies to the use of the "mpdv_calc" format and the classic industry minute display:

When converting from seconds to hours (division by 3600), decimal numbers with an infinite number of decimal places can occur, which inevitably have to be rounded when displayed on the client. Example: 20 minutes = 1200 seconds = 0.333333… hours. If the value is rounded to three decimal places, you calculate backward as follows: 0.333 * 3600 = 1198.8 seconds. Depending on how the client rounds, the internal value is then no longer 1200 seconds, but 1199 or 1198 seconds.

**InputFormat**

Input format to check user input. Normally, the MOC automatically defines the appropriate input format that matches the output format. You should only indicate the InputFormat if additional checks are required. So-called "regular expressions" specify the InputFormat. Regular expressions are standard in software development. You can find further information on regular expressions on the Internet.

**Length**

Field length: number of characters.

## Configuration of quantities

You can change the output and input formats for quantity fields:

**Output format**

n<x>: shows the number with thousands separator. <x> indicates the number of decimal places. e. g. n1, n2 … .

f<x>: shows the number without thousands separator. <x> indicates the number of decimal places. e. g. f1, f2 … .

**Input format (examples)**

[0-9]{0,10}

Integer with up to 10 digits. No minus sign allowed; only positive values supported.

-?[0-9]{0,10}

Integer with up to 10 digits and optional, leading minus sign.

-?[0-9]{0,9}\R.?[0-9]{0,3}

Optional sign, up to 9 places before decimal point and optionally up to three decimal places.

## Configuration of cycles

You can edit the label, UnitLabel, OutputFormat and length. Meaningful values are assigned by default to "UnitLabel" and "OutputFormat". The following configurations are useful:

| Unit Label | Output Format |
|---|---|
| lkHrsPer1000 | {0:mpdv_cycletime} |

| | |
|---|---|
| lkUnitsSecondsPerOne | {0:mpdv_cycletime_sec_cycle} |
| lkUnitPiecePerHour | {0:mpdv_cycletime_piece_hour} |

## Configuration of single piece specifications (te, teb)

- Syntactic types starting with "st_iw_“ affect incentive pay applications.
- Syntactic types starting with "st_mf_“ are used in all other applications.

You can edit the label, UnitLabel, OutputFormat and length. The MOC automatically assigns expedient values to the *InputFormat.*

You can use the format "mpdv_calc…“ to perform calculations for the output format. The basis of the calculation is the value available in the database, which is always in "seconds per 1000 pieces".

| **UnitLabel** | **OutputFormat** |
|---|---|
| lkHrsPer1000<br>= [h/1000] | {0:mpdv_te} |
| lkUnitsSecondsPerOne<br>= [sec/1] | mpdv_calc;MULT=1;DIV=1000;INVERSE=false;FORMAT=f3 |
| lkUnitMinutesPerPiece<br>= [min/1] | mpdv_calc;MULT=1;DIV=60000;INVERSE=false;FORMAT=f3 |
| lkUnitPiecePerHour<br>= [1/h] | mpdv_calc;MULT=1;DIV=3600000;INVERSE=true;FORMAT=f3 |
| [1/min] | mpdv_calc;MULT=1;DIV=60000;INVERSE=true;FORMAT=f3 |
| … | … |

Output formats with calculation allow you to calculate the inverse by setting "INVERSE=true". Consequently, you can display data as "quantity per time" instead of "time per quantity".

First use the multiplier and divisor. Then calculate the inverse. If you want to display the *pieces per minute,* you have to divide the $t_e$ in [sec/1000] by 60000 to get [*minutes/piece].* Then calculate the inverse to indicate [*pieces/minute].*

## Configuration of setup specifications (tr, trb)

- Syntactic types starting with "st_iw_“ affect incentive pay applications.
- Syntactic types starting with "st_mf_“ are used in all other applications.

You can edit the label, UnitLabel, OutputFormat and length. The MOC automatically assigns expedient values to the *InputFormat.*

You can use the format "mpdv_calc…“ to perform calculations for the output format. The value existing in the database is the basis for calculations. This value is stored in "seconds".

| UnitLabel | OutputFormat |
|---|---|
| lkHrs | {0:mpdv_te} |
| [min] | mpdv_calc;MULT=1;DIV=60;INVERSE=false;FORMAT=f3 |
| … | … |

## 3) Save changes

Click the "save" button to save changes to the local MOC client in the local scope. You should not change the file name.

## 4) Create update package

Click the "Create update package" button to start the Update Package Creator. The input fields are populated with default values. Enter the following settings:

**Path (folder that includes the generated update)**

Specify the folder where the update package is stored. The folder must exist already.

**Update name**

We recommend appending a unique ID to the update name. You can add date and time, for example: "SyntacticTypes20170726_1342“.

**Version number**

Optionally, you can indicate a version number that will be displayed in the Maintenance Manager.

## 5) Install update package

The created Update Package is installed like any other update using the Maintenance Manager.

## 6) Update MOC clients

Usually, the MOC updater automatically downloads the updates to the MOC clients. You can use the menu to search immediately for updates (Help --> Search for updates). Once all MOC clients have been updated, the changes to the syntactic types take effect.

## 1.6 Change the MOC logging

By default, the client log files are stored in the user directory of the Windows user who runs the MOC. The log files are stored in the following directory, if this has not been changed:

```
[LocalApplicationData]\MPDV\MOC\log\
```

### 1.6.1 Change the storage location of the log file

We recommend separating the log entries of different MOC instances in order to facilitate the failure analysis. To change the storage location, create a new file named "NLog.**user**.config“ or "NLog.**local**.config“ with the following content in the main directory of the affected MOC installation (e.g. `"C:\Program Files (x86)\MPDV\MOC“):`

```
<?xml version="1.0" encoding="utf-8"?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" autoReload="true">
  <variable name="logpath" value="${specialfolder:folder=ApplicationData}\MPDV\MOC\log" />
</nlog>
```

Change the path highlighted in red and enter your required storage location (e.g. `${specialfolder:folder=ApplicationData}\MPDV\MOC`**`TEST`**`\log`). Ensure that the local user has write access to this folder.

Use the file NLog.**user**.config for customizations that only apply to this specific workplace (client). Whereas you should use the file NLog.**local**.config to distribute the modifications to all workstations (clients) via the update package.

### 1.6.2 Change the log level

In some rare cases, you might have to increase the MOC log level. To do so, create the file "NLog.user.config" with the following content as described in the previous section:

```
<?xml version="1.0" encoding="utf-8"?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" autoReload="true" globalThreshold="Trace">
</nlog>
```

Once you have sent the log file generated with the increased log level, you should delete this file because the increased log level can have a negative effect on the performance.